Laboratory 03

Aperiodic Servers & Producer–Consumer Pipeline

Real-Time Operating Systems • M.Eng. ECE • KMUTNB

Platform	NXP FRDM-MCXN236 (ARM Cortex-M33 @ 150 MHz)
RTOS	FreeRTOS v10.6
Toolchain	ARM GNU Toolchain (`arm-none-eabi-gcc`) + Make
Estimated time	3–4 hours
CLO mapping	CLO 2 & CLO 3

1. Objectives

By completing this lab you will be able to:

1. Implement a **deferred-interrupt handler** that moves ISR work into a task context.
2. Implement a **Polling Server** pattern using a FreeRTOS software timer and a request queue.
3. Measure **aperiodic response time** under a periodic background load and compare Polling Server vs. direct task notification.
4. Build a **two-producer, one-consumer** queue pipeline and observe queue saturation.
5. Use an **event group** to coordinate a multi-condition synchronisation scenario.

2. Prerequisites

- ☐ Lab 02 complete — you can build, flash, and observe a FreeRTOS application.
- ☐ ARM GNU Toolchain and `pyocd` working.
- ☐ FRDM-MCXN236 board connected via MCU-LINK USB.
- ☐ SEGGER SystemView installed (needed for Part D).

3. Background

The course Task Model assumes **periodic** tasks with known execution times. Real systems also receive **aperiodic** events (button presses, UART interrupts, sensor alerts) that must be handled without jeopardising periodic task deadlines.

Two strategies are in scope for this lab:

Strategy	Mechanism	Response time	Periodic impact
Deferred interrupt	ISR gives semaphore → task unblocks immediately	Very low	Depends on handler task priority
Polling Server	Software timer wakes a service task on a fixed budget and period	Up to one server period	Bounded — server is treated as a periodic task

4. Project Structure

```
lab03_aperiodic_server/
  Makefile
  src/
    main.c                 <- creates all tasks and kernel objects
    periodic_tasks.c/.h    <- tau1 (100 ms), tau2 (200 ms) background load
    aperiodic_handler.c/.h <- deferred ISR task + Polling Server task
```

```
    producer_consumer.c/.h <- two-producer / one-consumer pipeline (Part C)
    FreeRTOSConfig.h
```

Note

The FreeRTOS kernel and all hardware drivers come from the NXP MCUXpresso SDK — no separate kernel clone is needed. Hardware initialisation (clocks, pin-mux, LPUART4) is handled by `BOARD_InitHardware()` from the SDK board-support package. Use `PRINTF` from `fsl_debug_console.h` for all console output.

4.1. Task and Kernel Object Map

Object	Type	Notes
`vPeriodic1Task`	Task prio 3, T=100 ms	Background periodic load
`vPeriodic2Task`	Task prio 2, T=200 ms	Background periodic load
`vAperiodicHandlerTask`	Task prio 4	Deferred ISR handler (Part A)
`vPollingServerTask`	Task prio 3	Budget-limited service task (Part B)
`xAperiodicSem`	Binary semaphore	ISR → handler signal
`xRequestQueue`	Queue depth 4	Polling Server request queue
`vSensor1Task`	Task prio 3, T=100 ms	Producer 1 (Part C)
`vSensor2Task`	Task prio 2, T=200 ms	Producer 2 (Part C)
`vLoggerTask`	Task prio 1	Consumer (Part C)
`xPipelineQueue`	Queue depth 10	Sensor → Logger (Part C)
`xPipelineEvents`	Event group	`DATA_READY` + `ALARM` bits (Part C)

5. Part A — Deferred Interrupt Handler

5.1. Step 1 — Wire the Button ISR

The FRDM-MCXN236 has a user button on GPIO P0_23. The ISR is already registered in `main.c`; your task is to implement the handler.

In `aperiodic_handler.c`, implement:

```c
/* ISR -- keep it minimal: signal and return */
void GPIO0_IRQHandler(void)
{
    BaseType_t xHigherPriorityTaskWoken = pdFALSE;
    GPIO_PinClearInterruptFlag(GPIO0, 23u);
    xSemaphoreGiveFromISR(xAperiodicSem, &xHigherPriorityTaskWoken);
    portYIELD_FROM_ISR(xHigherPriorityTaskWoken);
}

/* Handler task -- runs at prio 4, higher than all periodic tasks */
void vAperiodicHandlerTask(void *pv)
{
    uint32_t count = 0;
```

```
    for (;;) {
        xSemaphoreTake(xAperiodicSem, portMAX_DELAY);
        count++;
        PRINTF("[ASYNC] button press #%u -- handled in task context
            t=%u ms\r\n",
               count, (unsigned)xTaskGetTickCount());
    }
}
```

5.2. Step 2 — Build and Test

```
cd labs/lab03_aperiodic_server
mingw32-make
mingw32-make flash
```

Press the user button several times and observe the terminal. Each press should appear as an `[ASYNC]` line between the `[P1]` and `[P2]` periodic lines.

Checkpoint A

Terminal showing at least 3 button presses interleaved with periodic output, with no missed presses.

Question A1

Press the button rapidly 5 times in under 100 ms. Do any presses get lost? Why or why not? (*Hint:* `xAperiodicSem` is a binary semaphore. What happens to the second give if the first has not been taken yet?)

Question A2

The handler task runs at priority 4, above all periodic tasks. What is the cost of this choice? When would you set the handler priority *below* the periodic tasks?

6. Part B — Polling Server

6.1. Step 1 — Implement the Server

A Polling Server is a periodic task with a fixed budget B_s. It checks a request queue and drains up to B_s ms of work per activation.

In `aperiodic_handler.c`:

```
#define POLLING_SERVER_BUDGET_MS   10    /* max service per period */
#define POLLING_SERVER_PERIOD_MS   50    /* Ts */

void vPollingServerTask(void *pv)
{
    TickType_t xLastWake = xTaskGetTickCount();
    for (;;) {
        vTaskDelayUntil(&xLastWake, pdMS_TO_TICKS(
            POLLING_SERVER_PERIOD_MS));

        TickType_t budget_start = xTaskGetTickCount();
        AperiodicRequest_t req;

        while ((xTaskGetTickCount() - budget_start) <
               pdMS_TO_TICKS(POLLING_SERVER_BUDGET_MS))
        {
```

```
            if (xQueueReceive(xRequestQueue, &req, 0) != pdTRUE)
                break;
            PRINTF("[PS] served request type=%d  arrived=%u ms
                served=%u ms\r\n",
                    req.type, req.timestamp, (unsigned)
                        xTaskGetTickCount());
        }
    }
}
```

Note

The Polling Server is parameterised as a periodic task with $C_i = B_s$, $T_i = T_s$ for schedulability analysis. Add it to the utilisation calculation — does the total U stay below $\ln 2$?

6.2. Step 2 — Submit Requests from an ISR

Wire the button ISR to enqueue a request instead of giving a semaphore (switch `main.c` to `#define LAB_PART POLLING_SERVER`):

```
void GPIO0_IRQHandler(void)
{
    BaseType_t xHPTW = pdFALSE;
    GPIO_PinClearInterruptFlag(GPIO0, 23u);
    AperiodicRequest_t req = { .type = 1,
                               .timestamp = xTaskGetTickCountFromISR
                                   () };
    xQueueSendFromISR(xRequestQueue, &req, &xHPTW);
    portYIELD_FROM_ISR(xHPTW);
}
```

6.3. Step 3 — Observe and Measure

Press the button and note the time between the press (timestamp logged in the ISR) and the `[PS] served` line.

Checkpoint B

Terminal output with at least 5 requests served, annotated with arrival time and service time.

Question B1

What is the worst-case aperiodic response time of the Polling Server in terms of T_s? With $T_s = 50\,\text{ms}$, what is the maximum expected wait? Does your measured data agree?

Question B2

You pressed the button 8 times quickly. The queue depth is 4. What happened to the other 4 requests? What would change if you used `xQueueSendFromISR` with `portMAX_DELAY`?

Question B3

Compare the Polling Server with the direct deferred handler from Part A. Fill in this table from your observations:

Metric	Deferred handler	Polling Server
Response time (typical)		
Response time (worst case)		
Effect on periodic tasks at peak load		
Analysis complexity		

7. Part C — Two-Producer, One-Consumer Pipeline

7.1. Step 1 — Build the Pipeline

Implement in `producer_consumer.c`:

```c
typedef struct {
    uint8_t  sensor_id;
    int16_t  value;
    uint32_t timestamp;
} SensorReading_t;

void vSensor1Task(void *pv)   /* T=100 ms, prio 3 */
{
    static int16_t raw = 0;
    TickType_t xLastWake = xTaskGetTickCount();
    for (;;) {
        vTaskDelayUntil(&xLastWake, pdMS_TO_TICKS(100));
        raw = (raw + 7) % 1000;
        SensorReading_t r = { .sensor_id = 1, .value = raw,
                              .timestamp = xTaskGetTickCount() };
        if (xQueueSend(xPipelineQueue, &r, 0) != pdTRUE)
            PRINTF("[S1] queue full -- dropped!\r\n");
    }
}

void vSensor2Task(void *pv)   /* T=200 ms, prio 2 */
{
    static int16_t raw = 500;
    TickType_t xLastWake = xTaskGetTickCount();
    for (;;) {
        vTaskDelayUntil(&xLastWake, pdMS_TO_TICKS(200));
        raw = (raw + 13) % 1000;
        SensorReading_t r = { .sensor_id = 2, .value = raw,
                              .timestamp = xTaskGetTickCount() };
        xQueueSend(xPipelineQueue, &r, 0);
    }
}

void vLoggerTask(void *pv)    /* prio 1 -- lowest */
{
    static uint32_t count = 0;
    SensorReading_t r;
    for (;;) {
        xQueueReceive(xPipelineQueue, &r, portMAX_DELAY);
        count++;
        PRINTF("[LOG] #%u  sensor=%d  val=%d  t=%u ms\r\n",
               count, r.sensor_id, r.value, r.timestamp);
```

```
        if (r.value > 800)
            xEventGroupSetBits(xPipelineEvents, BIT_ALARM);
        if ((count % 10) == 0)
            xEventGroupSetBits(xPipelineEvents, BIT_DATA_READY);
    }
}
```

7.2. Step 2 — Add Event Group Coordination

A fourth task waits for `BIT_DATA_READY` (every 10 items processed) OR `BIT_ALARM` (value > 800):

```
#define BIT_DATA_READY    (1 << 0)
#define BIT_ALARM         (1 << 1)

void vMonitorTask(void *pv)    /* prio 2 */
{
    for (;;) {
        EventBits_t bits = xEventGroupWaitBits(
            xPipelineEvents,
            BIT_DATA_READY | BIT_ALARM,
            pdTRUE,    /* clear on exit */
            pdFALSE,   /* OR -- either bit suffices */
            portMAX_DELAY);
        if (bits & BIT_DATA_READY)
            PRINTF("[MON] 10-item batch complete  t=%u ms\r\n",
                   (unsigned)xTaskGetTickCount());
        if (bits & BIT_ALARM)
            PRINTF("[MON] ALARM: value exceeded threshold  t=%u ms\r
                \n",
                   (unsigned)xTaskGetTickCount());
    }
}
```

Set `BIT_ALARM` inside `vLoggerTask` when `r.value > 800`.

Checkpoint C

Terminal showing both `[MON] 10-item batch` and `[MON] ALARM` lines with correct timing.

Question C1

Why is the event group AND (`pdTRUE`) vs. OR (`pdFALSE`) distinction important here? What would happen if you used AND for this monitor task?

Question C2

Use `uxQueueMessagesWaiting(xPipelineQueue)` inside `vLoggerTask` and print it every 10 items. Does the queue ever have more than 1 item waiting? Explain why or why not at current production vs. consumption rates.

Question C3

Artificially slow down `vLoggerTask` with `vTaskDelay(pdMS_TO_TICKS(150))` after each print. Watch `[S1] queue full` messages appear. What is the queue high-water mark just before the first drop? How does this inform the minimum required queue depth for a 2-second burst?

8. Part D — SystemView Trace

Rebuild without the slowing `vTaskDelay` from Question C3. Capture a SystemView trace of the full system (all tasks running).

1. Open SystemView → Start Recording.
2. Run for at least 5 seconds.
3. Stop recording and export the trace.

In the trace identify and annotate:

- A context switch from `vLoggerTask` to `vSensor1Task` triggered by timer expiry.
- A `xQueueSend` call inside `vSensor1Task` that directly unblocks `vLoggerTask`.
- The `[MON]` task sleeping until the event group fires.

Checkpoint D

Annotated screenshot of the SystemView timeline.

9. Deliverables

#	Item	Details
1	Part A terminal capture	3+ button presses interleaved with periodic output
2	Part B comparison table	Question B3 with measured values
3	Part C terminal capture	Both event group triggers visible
4	Part D SystemView screenshot	Annotated timeline
5	Written answers	Questions A1–A2, B1–B3, C1–C3
6	`FreeRTOSConfig.h`	Utilisation calculation comment including Polling Server

10. Key Concepts Introduced

Concept	Where you saw it
Deferred interrupt handler pattern	Part A
Binary semaphore for ISR→task signalling	Part A
Polling Server budget and period	Part B
Aperiodic response time under periodic load	Parts A & B
Multi-producer queue pipeline	Part C
Queue saturation and flow control	Part C
Event group AND vs. OR synchronisation	Part C

11. Implementation Notes

11.1. GPIO Interrupt Flag API (SDK v2.x)

The current NXP MCUXpresso SDK uses a per-pin API to clear GPIO interrupt flags. Use `GPIO_PinClearInterruptFlag` instead of the old mask-based function:

```
/* Correct -- current SDK (MCUXpresso SDK 2.x) */
```

```
GPIO_PinClearInterruptFlag(GPIO0, 23u);   /* pin number, not bit
    mask */

/* Old API -- removed from current SDK, will fail to link */
/* GPIO_ClearPinsInterruptFlags(GPIO0, 1u << 23); */
```

The function prototype is declared in `fsl_gpio.h`:

```
void GPIO_PinClearInterruptFlag(GPIO_Type *base, uint32_t pin);
```

12. Reference

Resource	Relevant sections
Barry, *Mastering the FreeRTOS Kernel*	Ch. 6 (semaphores), Ch. 8 (event groups)
Buttazzo, *Hard Real-Time Computing Systems*	Ch. 5 (aperiodic task scheduling)
FreeRTOS docs	`xSemaphoreGiveFromISR`, `xQueueSendFromISR`, `xEventGroupSetBits`, `xEventGroupWaitBits`